

Understanding Attestation

Node Attestation · Runtime Attestation · Direct Anonymous Attestation

A plain-English technical guide covering how modern systems prove their identity and integrity — from basic device attestation all the way to privacy-preserving anonymous attestation using TPM 2.0 hardware.

Topics covered: Node Attestation · Runtime Attestation · Direct Anonymous Attestation · TPM 2.0 · Zero Trust

Contents

1	What Is Attestation?	3
2	The Entities in Attestation	4
3	Node Attestation — Proving the Device Is Real	5
4	Runtime Attestation — Proving the Software Is Clean	7
5	The Privacy Problem with Traditional Attestation	9
6	Direct Anonymous Attestation (DAA)	10
7	The UBITECH DAA Library	12
8	How DAA Fits Into Zero Trust	14
9	Terminology Cheat Sheet	15

1. What Is Attestation?

When you connect to a bank's website, your browser checks the bank's certificate to confirm it is talking to the real bank and not an impostor. That is identity verification for a server. **Attestation** is the same idea applied to an entire device — it is a mechanism by which a device proves not just its identity, but also the integrity of what is running on it.

A device that supports attestation can answer two fundamental questions:

- **Am I a legitimate, genuine device?** (Node Attestation)
- **Has my software been tampered with?** (Runtime Attestation)

Think of attestation like a health certificate for a device. Just as a food safety inspector checks that a kitchen is clean and operating correctly before allowing it to serve food, attestation checks that a device is genuine and unmodified before allowing it to access sensitive systems.

Why It Matters

Modern computing environments — cloud infrastructure, enterprise networks, IoT deployments, and confidential computing platforms — all face the same threat: how do you know the device on the other end of a connection is trustworthy? A compromised device can leak secrets, forward data to attackers, or impersonate legitimate services. Attestation provides a cryptographic answer to this question, grounded in hardware that is difficult to tamper with.

The Hardware Anchor: The TPM

Almost all attestation in practice depends on a **Trusted Platform Module (TPM)** — a small, dedicated security chip built into modern computers and servers. The TPM holds cryptographic keys in protected storage that cannot be read or copied by software, even by the operating system or a privileged attacker. It also silently records hashes of everything that boots on the machine. These two capabilities — hardware-protected keys and tamper-evident measurements — are the foundation on which all attestation is built.

2. The Entities in Attestation

Multiple parties are involved in any attestation exchange. The IETF RATS (Remote ATtestation procedureS) architecture, defined in RFC 9334, gives these parties standard names. Different vendors use different terms for the same roles — a cheat sheet is provided in Chapter 9.

Entity	What It Does	Real-World Example
Attester	The device being evaluated. It gathers evidence about itself and sends it for appraisal.	Your laptop, a cloud VM, an IoT sensor
Verifier	Receives the evidence and checks it against known-good values. Produces a verdict.	Microsoft Azure Attestation, AWS Nitro Attestation
Relying Party	The service that wants to do business with the device. It waits for the Verifier's verdict before granting access.	A web application, an API gateway, a database
Endorser	Typically the hardware manufacturer. Vouches for the hardware at factory time by signing its certificates.	Intel, AMD, ARM, a TPM manufacturer
Reference Value Provider	Maintains a database of known-good software measurements. The Verifier compares against these to detect tampering.	A vendor's firmware update server, a corporate IT baseline database

How They Work Together

The flow is straightforward: the **Endorser** vouches for the hardware once (at manufacture). The **Reference Value Provider** maintains what healthy software looks like. At attestation time, the **Attester** gathers its evidence and sends it to the **Verifier**, which compares it against both the Endorser's certificates and the Reference Values. The Verifier issues a verdict, and the **Relying Party** acts on it.

Airport security analogy: The Endorser is the passport office (vouched for your identity document before you arrived). The Attester is you at the gate with your passport. The Verifier is the security officer checking documents. The Reference Value Provider is the database of valid flights and IDs. The Relying Party is the airline — it only lets you board once security clears you.

3. Node Attestation

Proving the Device Is Real

Node attestation answers a single question: **Is this a genuine, legitimate device?** It does not care about what software is running — that is runtime attestation's job. Node attestation focuses purely on identity: proving that the hardware is real and has not been spoofed.

The Endorsement Key (EK)

Every TPM is manufactured with a unique cryptographic key pair called the **Endorsement Key (EK)**. The private half never leaves the TPM — it is physically impossible to extract it. The public half is signed by the TPM manufacturer (the Endorser) into a certificate, the **EK Certificate**, which is burned into the device at the factory.

This certificate is the hardware's birth certificate. When a verifier wants to confirm a device is genuine, it checks the EK Certificate chain back to the manufacturer's root certificate. If the chain is valid, the device is genuine.

The Attestation Key (AK)

The EK is a long-term identity key — using it directly for every attestation would create a permanent, traceable identity. Instead, the TPM generates a separate **Attestation Key (AK)** for signing attestation data. The AK is cryptographically bound to the EK (proving it came from the same TPM), but it can be rotated or replaced without changing the device's fundamental identity.

1	Device generates an Attestation Key (AK) inside the TPM. The private key never leaves the chip.
2	Device sends the AK public key and EK Certificate to a Provisioning Service. This is a one-time enrollment step.
3	Provisioning Service verifies the EK Certificate against the manufacturer's root. Confirms the TPM is genuine hardware.
4	Provisioning Service certifies the AK — confirming it came from a genuine TPM. Issues an AK Certificate.
5	At attestation time, the device signs a challenge using the AK. The Verifier checks the AK Certificate to confirm the signature is from a real TPM.

What Node Attestation Does NOT Cover

Node attestation only proves that the hardware is genuine. It says nothing about what software is installed or running. A legitimate TPM on a fully compromised machine will still pass node attestation. For software integrity, you need runtime attestation — covered in Chapter 4.

Real-World Deployments

Node attestation underpins several well-known technologies. Windows uses it in Device Health Attestation to verify that a machine accessing corporate resources has a genuine TPM. Cloud providers use it to confirm that virtual machines are running on real hardware before issuing certificates. FIDO2 security keys use a similar concept to prove the authenticator is a genuine hardware device.

4. Runtime Attestation

Proving the Software Is Clean

Runtime attestation answers a different question: **Has the software on this device been tampered with?** Even if you have confirmed the hardware is genuine (node attestation), an attacker who has compromised the OS or modified a firmware component could still cause harm. Runtime attestation detects this.

PCR Registers — The TPM's Tamper-Evident Log

The TPM contains a set of registers called **Platform Configuration Registers (PCRs)**. These are special — you can only extend them, never write to them directly. Each time a component boots (firmware, bootloader, OS kernel, drivers), it measures itself by hashing its own code and extending that hash into a PCR. The result is a chain of measurements that represents the entire boot sequence.

Extending a PCR works like this: $\text{new_PCR_value} = \text{Hash}(\text{old_PCR_value} + \text{new_measurement})$. This means every measurement is dependent on everything that came before it. You cannot remove or alter an earlier measurement without invalidating all subsequent ones. The chain is tamper-evident.

What Gets Measured

PCR	What Is Measured
PCR 0	BIOS/UEFI firmware
PCR 1	BIOS/UEFI configuration
PCR 2-3	Option ROMs and drivers
PCR 4	Boot manager (e.g. GRUB, Windows Boot Manager)
PCR 5	Boot manager configuration
PCR 7	Secure Boot policy and keys
PCR 11+	OS kernel, initrd, kernel command line

Quotes — Signed PCR Snapshots

A **Quote** is the mechanism by which the TPM reports its PCR values in a tamper-proof way. The device asks its TPM to sign a snapshot of specified PCR values using the Attestation Key. Because the AK private key never leaves the TPM, a valid quote proves that those PCR values came from that specific TPM and have not been fabricated by software.

The Runtime Attestation Flow

1	Verifier sends a nonce (random challenge) to the Attester. The nonce ensures the quote is fresh and not a replay.
2	Attester asks its TPM to sign the current PCR values along with the nonce. The TPM produces a Quote signed by the AK.
3	Attester sends the Quote and the event log to the Verifier. The event log explains what each PCR extension corresponded to.
4	Verifier checks the Quote signature using the AK Certificate. Confirms the values genuinely came from a real TPM.
5	Verifier replays the event log and checks final PCR values match known-good baselines. If they match, the software stack is unmodified. If not, tampering is detected.

Important limitation: Runtime attestation only covers the boot-time software stack. If malware is injected after the OS boots (a so-called live attack), PCRs will not reflect it unless additional runtime measurement mechanisms (like Linux IMA — Integrity Measurement Architecture) are in place.

5. The Privacy Problem with Traditional Attestation

Node attestation and runtime attestation together give a strong picture of whether a device is genuine and clean. But they introduce a serious privacy problem: **every attestation reveals a unique device identity**.

When a device sends its EK Certificate or signs with its AK, any service that receives that signature can identify the specific device. Worse, multiple services can share their observations and build a profile of where that device has been, what services it accesses, and when. This is device-level surveillance.

Concrete example: A hospital deploys 500 medical devices. Each attests to a central service before uploading patient data. If the attestation uses a unique device identity, the service can track exactly which device handled which patient at what time — even if the data itself is anonymized. For regulated industries (GDPR, HIPAA), this is a compliance problem as much as a technical one.

The Dilemma

Traditional attestation forces a choice: you can have strong hardware-rooted security, or you can have user privacy, but not both. Direct Anonymous Attestation was invented specifically to break this dilemma.

6. Direct Anonymous Attestation (DAA)

Direct Anonymous Attestation (DAA) is a cryptographic protocol that allows a device to prove it is a legitimate, genuine TPM-equipped machine — without revealing which device it is. Security and privacy at the same time.

The Core Idea

DAA works as a **group signature scheme**. A trusted authority (the Issuer) enrolls a group of legitimate devices and gives each one a credential. When a device needs to attest, it signs using that credential. The Verifier can confirm the signature came from a member of the approved group — but cannot determine which specific member signed.

Analogy: Imagine a country issues anonymous voting tokens. Each citizen gets one token that proves they are an eligible voter, but the token itself contains no name or ID. When you cast a vote, the polling station can confirm you have a valid token, but cannot connect the vote to you personally. DAA works on exactly this principle, but for devices and cryptographic proofs.

The Three Entities

Entity	What It Does	Real-World Example
Issuer	Verifies that a device has a genuine TPM, then issues an anonymous group credential. Runs an enrollment service — not involved in day-to-day attestation.	The chip manufacturer, a corporate IT authority, or a cloud provider
Member (Attester)	Holds the DAA credential (locked in the TPM). Randomizes it to produce a fresh one-time signature for each attestation — unlinkable across sessions.	Your device: laptop, server, IoT node
Verifier	Checks the signature against the Issuer's public group key. Confirms the device is a genuine group member. Cannot identify which device signed.	A web service, API gateway, or enterprise access system

The Two Protocols

Protocol 1 — Join (Enrollment, done once)

1	TPM generates a secret key f and locks it inside the chip. The key never leaves the TPM — not even the OS can read it.
2	Device generates a cryptographic commitment to f. A commitment proves you know f without revealing f .

3	Device sends the commitment and a zero-knowledge proof to the Issuer. The proof mathematically demonstrates possession of a genuine TPM key.
4	Issuer verifies the proof and issues a DAA credential. The credential is a blind signature — the Issuer does not learn f.
5	Device stores the credential. Enrollment is complete. This step happens once.

Protocol 2 — Sign (Attestation, done every time)

1	Verifier sends a challenge to the device. Usually a random nonce to prevent replay attacks.
2	Device asks the TPM to randomize its credential. The TPM produces a fresh randomization — the output is different every single time.
3	Device constructs a DAA signature using the randomized credential. The signature proves group membership without revealing the credential itself.
4	Device sends the signature to the Verifier. No two signatures from the same device are linkable.
5	Verifier checks the signature against the Issuer's group public key. Valid: device is a legitimate group member. Identity: unknown.

Basename — Optional Pseudonymity

DAA supports an optional **basename**: a fixed string associated with a particular context (e.g., a service name). When a basename is used, the device produces a deterministic pseudonym for that context. Two signatures from the same device to the same service will share a pseudonym — allowing the service to detect a returning device or flag a rogue — but the pseudonym reveals nothing about the device's real identity or its interactions with other services.

7. The UBITECH DAA Library

The UBITECH DAA Library is an open-source C implementation of the DAA protocol targeting TPM 2.0 platforms. It was released in August 2025 by UBITECH's Secure Systems and Trusted Computing Research Group and is the reference implementation cited in the active IETF RATS DAA working group draft.

What It Implements

The library covers the full DAA protocol: the Join protocol (enrollment), the DAA-Signing protocol (anonymous attestation), and the Verification step. It is not a complete attestation solution — it is the cryptographic building block that enables the anonymous identity layer.

Source Files and Their Roles

File	Purpose
daa.c / daa.h	Core DAA logic — join, sign, verify protocols
issuer.c / issuer.h	Issuer-side key generation and credential issuance
tpm_funcs.c / tpm_funcs.h	TPM command wrappers using IBM TSS
cryptoutils.c / cryptoutils.h	RNG (CTR-DRBG), elliptic curve helpers, big integer operations
templates.c / templates.h	Data structure templates and serialization helpers
main.c	Example entry point demonstrating the full protocol flow
ibmtpm1682/	IBM TPM 2.0 simulator source — enables testing without hardware
cmake/	CMake modules pulling in Mbed TLS, MIRACL Core, and IBM TSS

Cryptographic Foundations

The library uses three cryptographic backends working together:

Mbed TLS — handles big integer arithmetic, standard ECC operations, and the CTR-DRBG random number generator seeded from system entropy (`/dev/urandom`). The functions `random_bytes` and `random_bytes_mod` wrap this.

MIRACL Core — provides pairing-based ECC on the BNP256 (Barreto-Naehrig) curve. This is the curve mandated by the TPM 2.0 standard for DAA and the same curve used by Intel EPID 2.0. Bilinear pairings are required for the credential structure of the DAA scheme.

IBM TPM 2.0 Simulator + TSS — provides the TPM command layer. The secret *f*-values are generated and held inside the TPM boundary. The host only ever sees commitments, never the raw secret material.

Building and Running

The library is built with CMake on Linux. A convenience script handles all dependencies:

```
# Quick build ./build.sh # Manual build mkdir -p build && cd build && cmake  
.. && make -j # Run against the IBM TPM simulator ./build/daa <tpm_port> #  
Run tests ./test.sh
```

Current Limitations

The library is a research and prototyping foundation — not a production-ready drop-in. Key limitations to be aware of: No credential revocation (on the roadmap). Linux only — no Windows or macOS support. No language bindings — pure C only (Python/Rust/Go bindings planned). Constant-time behavior is not guaranteed throughout — this matters for production security-critical paths and should be audited before deployment.

8. How DAA Fits Into Zero Trust

Zero Trust is a security model built on one principle: trust nobody by default, verify everything, every time. There is no concept of a trusted inside the network. Every request — from any device, user, or service — must continuously prove it is legitimate before being granted access.

Traditional security worked like a castle and moat: once inside the perimeter (VPN, office network, behind the firewall) you were trusted. Zero Trust eliminates the perimeter entirely. It requires:

- Continuous identity verification for users
- Continuous device health verification — this is where attestation comes in
- Least-privilege access: only the minimum permissions needed for each specific action
- Micro-segmentation: even inside the network, access to each resource is individually controlled

Where DAA Adds Value

In a Zero Trust architecture, attestation provides the device health check. The Verifier (or Policy Engine, in Zero Trust terminology) continuously checks that devices are genuine and unmodified before allowing access. This is necessary but traditionally creates a surveillance problem: every check reveals device identity, building a complete record of every device's movements and access patterns.

DAA allows the device health check to happen continuously — satisfying Zero Trust requirements — without creating that surveillance record. Each check is anonymous and unlinkable. The Verifier knows the device is clean; it learns nothing else.

In a Zero Trust architecture, attestation is the device health check layer. DAA takes this further by making those checks continuous and anonymous — the Verifier knows a device is clean and legitimate, but learns nothing about which specific device it is or what other services it has accessed.

9. Terminology Cheat Sheet

The same role, many names

Different standards bodies, vendors, and frameworks use different terms for the same roles. This table maps them.

Plain English	IETF RATS	Zero Trust / NIST	OAuth2 / OIDC	SAML / SSO
The device being checked	Attester	Endpoint	Client	Subject
The thing that checks it	Verifier	Policy Engine	Authorization Server	Identity Provider
The service using the result	Relying Party	Policy Enforcement Point	Resource Server	Service Provider
The hardware voucher	Endorser	Trust Anchor	—	—
The known-good database	Reference Value Provider	Policy Database	—	—

Key Technical Terms

TPM — Trusted Platform Module — a tamper-resistant security chip that stores keys and records measurements.

EK — Endorsement Key — the TPM's unique, factory-burned identity key. The private half never leaves the chip.

AK — Attestation Key — a signing key generated by the TPM, used for day-to-day attestation instead of the EK.

PCR — Platform Configuration Register — a TPM register that records a tamper-evident chain of software measurements.

Quote — A TPM-signed snapshot of PCR values — proves the measurements are genuine and came from a real TPM.

DAA — Direct Anonymous Attestation — a group signature scheme enabling anonymous device attestation.

EK Certificate — The manufacturer-signed certificate for the EK, proving the TPM is genuine hardware.

Zero-Knowledge Proof — A cryptographic proof that demonstrates knowledge of a secret without revealing the secret itself.

Basename — An optional context string in DAA that produces a deterministic pseudonym per device per service.

BNP256 — Barreto-Naehrig P-256 — the elliptic curve used by TPM 2.0 for DAA operations.

Standards references: IETF RFC 9334 (RATS Architecture) | TCG TPM 2.0 Library Specification | ISO/IEC 20008 (DAA) | NIST SP 800-155